ORCHESTRATION-DESIGN · WORKED EXAMPLE

200 restaurants, 3,000 reviews a week

A real problem, run through the skill end to end. It's the first example where the ladder **climbs** — and the interesting part is what nearly went wrong on the way up.

THE PROBLEM, AS AN OPERATIONS DIRECTOR WOULD SAY IT

> We get about 3,000 customer reviews a week across 200 locations. The dashboard shows me star ratings, which tell me *that* a location is struggling and nothing about *why*. Nobody has time to read them. I want to know what's actually going wrong at each restaurant.
>
> THE REQUEST

What most people build for this: a multi-agent system. A sentiment agent, a categoriser agent, a summariser agent, a report agent — four boxes in a row, one per verb in the sentence. It looks like engineering.

The skill starts somewhere else entirely: with two numbers.

PHASE 0 — THE TWO NUMBERS THAT DECIDE EVERYTHING

A WEEK OF RAW REVIEWS	ALL 200 SUMMARIES	PER LOCATION
240,000	**30,000**	**15**
tokens. 3,000 reviews × ~80. **Exceeds a single context window.**	tokens. 200 × ~150. **Fits comfortably.**	reviews a week — small enough that one location is easy, and 200 of them is not.

The raw reviews force a split. The summaries do not. That gap is the entire design, and it was invisible until someone multiplied two numbers.

The failure that hurts

Not a slow run, and not cost. It's this: **a real problem at one location gets averaged away.** Summarise 3,000 reviews into "food quality down 3%" and the single restaurant with a pattern of food-safety complaints disappears into the mean. Missing a signal is far worse here than taking an extra hour.

That answer determines where the reviewer goes, what the bounds protect, and what the exhaustion terminal has to say out loud. One Phase 0 question, and it shapes four later decisions.

PHASE 1 — WALKING THE LADDER

1 · script	CLIMB	"Waited 40 minutes for cold food" is a *kitchen throughput* problem, not a food-quality tag. Causal categorisation is judgement a keyword rule cannot encode.
2 · loop	CLIMB	Whether a location summary is *accurate* cannot be asserted mechanically. Schema validity can; truth cannot.
3 · + reviewer	CLIMB	A reviewer helps and is kept — but it doesn't solve the real constraint. 240k tokens still don't fit.
4 · panel	NO	One defect class matters: an invented issue with no review behind it. One lens covers it.
5 · fan-out	STOP — TRIGGER TRUE	**The work exceeds one context window.** Not "the locations are independent" — they are, but independence is a precondition, never a trigger. The 240,000 is the trigger.
6 · durable	NOT YET	Weekly batch. Worth it when a mid-run crash starts costing a week — say so then, and only then.

THE TRAP ON THE WAY UP

Here is where a naive fan-out quietly ruins the product. Split the work 200 ways and each branch sees exactly one restaurant. Now ask the question the director actually cares about:

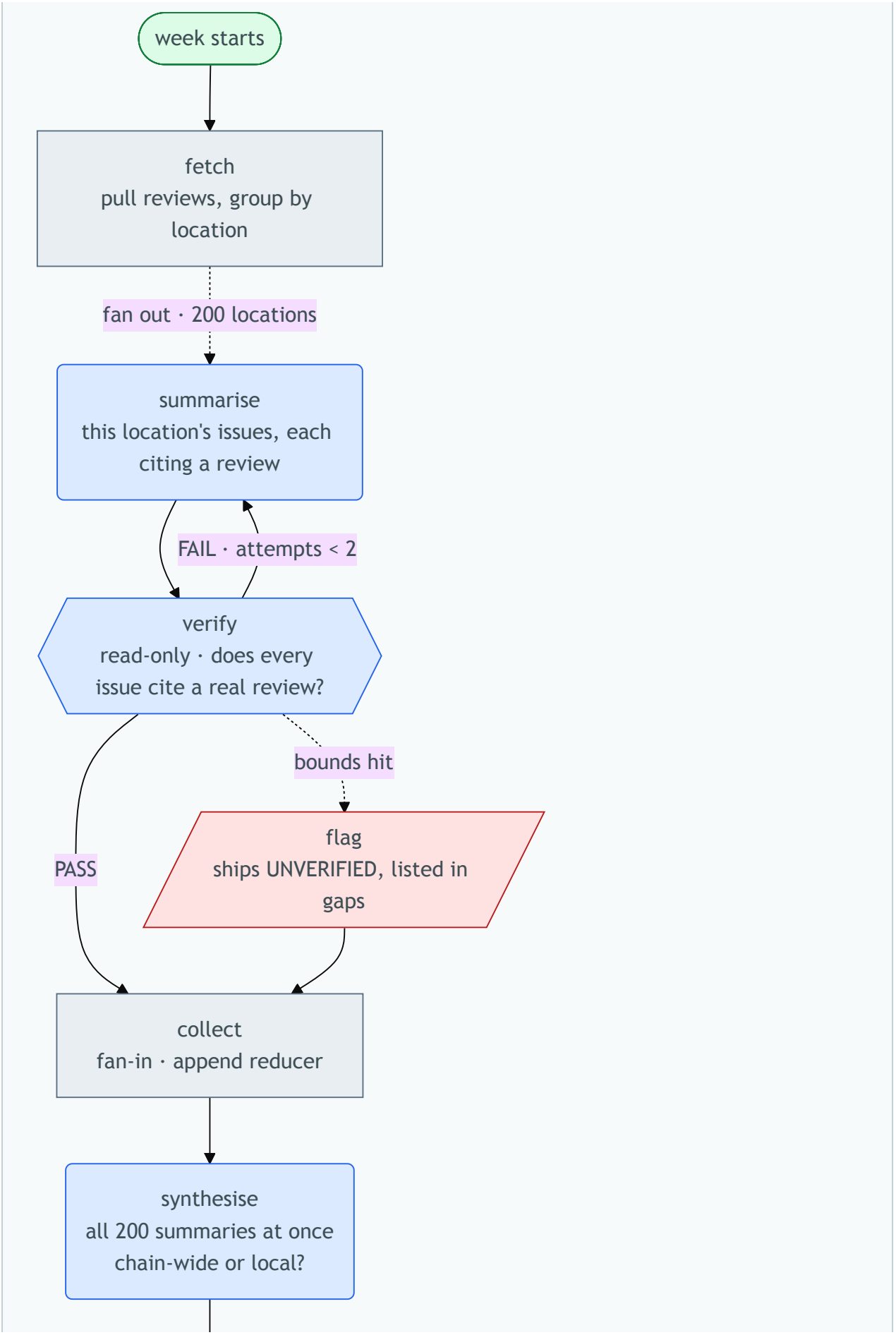
> Is this a problem with *this* restaurant, or is it happening everywhere?
>
> THE QUESTION FAN-OUT DESTROYS

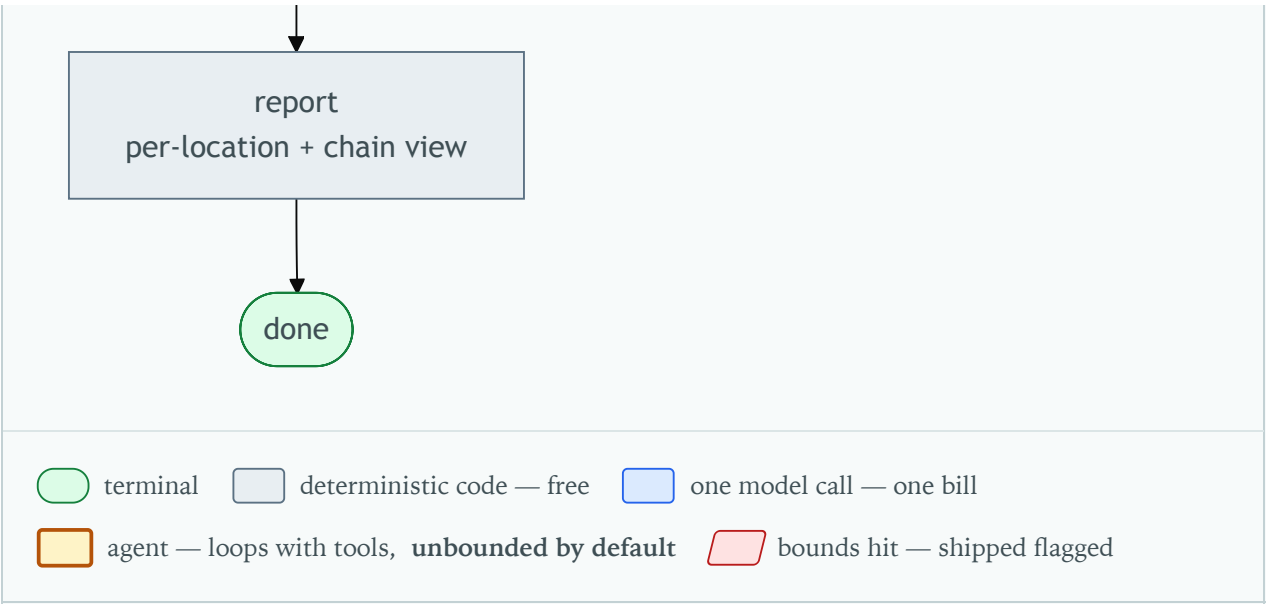
No branch can answer it. Cross-location judgement — is this chain-wide or local, which sites share a pattern, what's new this week — becomes structurally impossible the moment each agent sees one item. The skill names this cost explicitly: *any judgement needing to see items together is destroyed once each branch sees one item.*

Which is exactly why Phase 0 measured the summaries too. **30,000 tokens fits.** So the design fans out over the raw reviews, then brings all 200 summaries back into one context for a node that can compare them. The split happens where it's forced and nowhere else.

PHASE 2.5 — THE DESIGN, AS THE HUMAN SEES IT

FILL ENCODES COST AND RISK · NODE IDS ARE THE FUNCTION NAMES


week starts
fetch
pull reviews, group by location
fan out · 200 locations
summarise
this location's issues, each citing a review
FAIL · attempts < 2
verify
read-only · does every issue cite a real review?
bounds hit
PASS
flag
ships UNVERIFIED, listed in gaps
collect
fan-in · append reducer
synthesise
all 200 summaries at once
chain-wide or local?



Notice there is no amber. That absence is information: nothing in this design loops with tools until it decides it's finished, so nothing here can run away inside a bound that looks safe from outside. A design with an amber box would need its own iteration cap declared before anyone built it. This one doesn't — and you can see that in two seconds instead of reading three tables.

STATE — ONLY THE FIELDS THAT NEED IT

The skill's rule: table a field only if more than one thing writes it. Four of these have a single owner and get one line. One does not, and it's the one that silently loses data.

FIELD	OWNER	REDUCER	WHY
`reviews_by_location`	fetch	`replace`	Written once
`summaries`	**200 parallel branches**	**`append`**	**The one that matters.** A replace reducer here means branches overwrite each other and you get one summary where you expected 200 — with no error, ever
`errors`	any branch	`append`	Concurrent writers, same reason
`chain_findings`	synthesise	`replace`	Single owner
`tokens_spent`	every model node	`sum`	Read by the bound

BOUNDS AND COST

BOUND	VALUE
Re-summarise attempts	`2 per location`
Concurrency	`8 branches`
Spend	`token budget, read by the gate`
Exhaustion terminal	ships **marked UNVERIFIED** and appears in a gaps list — never silently

OPTION	PER RUN	BUYS
Level 3, one agent	`~240k`	Doesn't fit context. Degrades
Level 5 — chosen	**`~702k`**	**≈ $2.11/week.** Coverage that holds
Level 6, durable	`same`	Crash survival. Not yet

Level 5 costs nearly three times level 3 — and it's still right. That's the honest shape of this decision. Climbing wasn't an optimisation, it was forced: the cheaper option cannot do the job at all. The cost table exists so you say that out loud instead of discovering it in a billing alert.

THE BRIDGE TO CODE

Every node id in that diagram is a callable in the implementation, and every callable that owns state is a node in the diagram. That one rule is what stops the picture being decoration:

```
flowchart TD                          # the design
    fetch      ──────────────────▶    def fetch(cfg) -> dict:
    summarise  ──────────────────▶    def summarise(loc) -> dict:
    verify     ──────────────────▶    def verify(loc, summary) -> dict:
    collect    ──────────────────▶    def collect(state) -> dict:
    synthesise ──────────────────▶    def synthesise(state) -> dict:
    report     ──────────────────▶    def report(state) -> dict:

# not a comment. not an inline .invoke() buried in a loop body.
# a named thing you can grep for — which is the only correspondence
# that exists at levels 1–3, where no framework emits a topology.
```

The repo's own worked example was breaking this rule until recently: its diagram showed four nodes while the code had two functions and two inline model calls. The picture described something that didn't exist. Extracting them was clean — because the design had been right all along and only the code had drifted.

PHASE 4 — WHAT GETS ASSERTED

Not "does it look right". Five checks, each forcing the condition it guards:

#	ASSERTION	HOW IT'S FORCED
1	The reviewer is read-only	Snapshot the summary, run `verify`, assert it's byte-identical
2	Every bound is live	Substitute a verifier that never passes; assert the run stops at 2 attempts
3	Exhaustion is reachable *and marked*	Assert that run emits `UNVERIFIED` and a gaps entry
4	Failure is isolated	Break one location's fetch; assert the other 199 still complete
5	**The counts add up**	**`len(summaries) + len(errors) == 200`.** Nothing else notices silent parallel loss — the run looks fine and the data is wrong

WHAT THIS EXAMPLE IS ACTUALLY DEMONSTRATING

- **The gate isn't anti-structure.** Every previous run of this skill landed at level 1 or 3. Here the trigger was literally true and it climbed to 5 without argument — then designed the fan-out properly, with an append reducer and a count assertion.
- **Phase 0 did the real work.** Two multiplications — 3,000 × 80 and 200 × 150 — decided the level, located the split, and exposed the trap. No architecture discussion produced that; arithmetic did.
- **The method caught what fan-out breaks.** A naive parallel design would have shipped and silently lost the ability to answer the director's actual question. The synthesis node exists because the skill makes you name what splitting costs.
- **The diagram is checkable.** Colours say where money and risk are. Node names say what the functions will be called. Neither is decoration.

And the thing that would have shipped instead — sentiment agent, categoriser agent, summariser agent, report agent — would have cost more, fit no better, and answered a narrower question.

Worked example produced by running the installed `orchestration-design` skill on a stated problem, 3 Aug 2026. Diagram rendered and verified in a browser — 9 nodes, 10 edges, every class applied. Token figures are arithmetic from the stated volumes, not measurements. The design stops at Phase 2.5, which is a hard stop by construction: no implementation until a human has cut a node.
